Question 61

04: Prompt Engineering & Context

A developer wants Copilot to generate a custom date-parsing function that converts unconventional log timestamps into ISO-8601 strings. Which prompting strategy yields the highest first-pass accuracy?

A. Writing a single-word comment: `// parse date`.

✔ **B. Providing a descriptive function signature alongside 2–3 concrete input-to-output example pairs in a comment block (Few-Shot prompting).**

C. Closing all open tabs and deleting existing test files.

D. Running `gh copilot explain` on an empty directory.

Correct Answer: B

Supplying explicit **input/output examples** (Few-Shot prompting) combined with clear parameter/return type annotations removes ambiguity around edge cases and custom formatting rules, enabling Copilot to infer the exact transformation logic.

Option Breakdown

- **A. Single-word comment:** Incorrect. Vague prompts lack format specifications and force the model to guess.
- **B. Descriptive signature + input/output examples:** Correct. Demonstrates exact expected behavior for non-standard formats.
- **C. Closing tabs and deleting tests:** Incorrect. Removing relevant project context degrades completion relevance.
- **D. Running gh copilot explain:** Incorrect. The CLI explain command describes existing shell commands rather than generating application functions.

> 💡 **GH-300 Exam Tip**
> Whenever a prompt involves custom formatting, regex patterns, or data transformations, providing **2–3 input/output pairs** is one of the highest-leverage prompt engineering techniques tested on GH-300.